Comparing One-Stage Detectors for Maritime Target Detection on USVs

***Abstract*—This paper compares four single-stage detectors: YOLOv8, YOLO11, YOLO26 and SSD300, for maritime target detection from an unmanned surface vehicle. Experiments use MODD2, acquired by a real USV and annotated by experts. Models pre-trained on COCO are evaluated before and after domain adaptation by transfer learning. Adaptation raises F1 by 0.265 to 0.493, driven by a three- to five-fold increase in recall. YOLO11 attains the highest mAP@50 (0.698) and F1 (0.745) among the four architectures, though YOLO26 does not surpass it on any detection metric despite being the most recent model. Inference latency varied considerably across measurement sessions on cloud GPUs, so the accuracy–latency trade-off for embedded deployment is left for future hardware-specific evaluation. Recall broken down by object size shows the ranking is decided by targets below 0.05% of the image, which account for 72% of the test set.**

***Index Terms*—target detection, unmanned surface vehicle, YOLO, transfer learning, small target detection**

I. Introdução

Os Veículos de Superfície Não Tripulados (VSNT) são embarcações capazes de operar sem tripulação a bordo, por controle remoto ou por sistemas autônomos apoiados em sensores embarcados [1]. Sua modularidade vem consolidando-os como meios multipropósito para as marinhas modernas, em missões de vigilância, guerra de superfície e segurança marítima [2]. No caso da Marinha do Brasil, a motivação liga-se à extensão da Amazônia Azul, cuja vigilância contínua por meios exclusivamente tripulados demanda elevado contingente e expõe tripulações a riscos desnecessários [2]. Os sistemas tradicionais não suprem essa lacuna sozinhos: radares costeiros discriminam mal alvos de pequeno porte, e o Sistema de Identificação Automática depende da cooperação voluntária das embarcações, justamente o que falta às que operam de forma irregular [3]. Sensores visuais embarcados surgem, portanto, como complemento essencial [3], [4].

A detecção de embarcações e alvos flutuantes é a primitiva de percepção da plataforma: sem ela não há desvio de colisão, rastreamento nem classificação de alvo. Um detector genérico aplicado sem adaptação, porém, não resolve o problema. Modelos pré-treinados em conjuntos de propósito geral aprendem embarcações a partir de fotografias tomadas em terra, em ângulos e escalas distantes da vista de uma câmera ao nível do mar, onde o alvo ocupa poucas dezenas de pixels e se confunde com reflexos na água. Essa diferença de domínio motiva uma etapa de adaptação.

Este trabalho compara quatro arquiteturas de detecção de estágio único: YOLOv8, YOLO11, YOLO26 e SSD300, na detecção de embarcações em imagens capturadas por um VSNT real. As três primeiras representam a evolução recente da família YOLO [5], [6]; o SSD300 [7] entra como linha de base histórica. Todas partem de pesos do COCO [8] e são avaliadas antes e depois da adaptação por *transfer learning* [9] sobre o MODD2 [10]. A comparação considera as principais métricas de detecção e o custo de inferência, os fatores que determinam a viabilidade de embarcar o modelo. A Seção II revisa os trabalhos relacionados; a Seção III detalha a metodologia; a Seção IV apresenta os resultados; e a Seção V conclui o artigo.

II. Trabalhos Relacionados

A detecção de embarcações em ambiente marítimo por visão computacional é tratada em [3] e [4], que destacam os desafios impostos por reflexos da água, variações de iluminação e dinâmica do mar. Em [3], a combinação de redes convolucionais eficientes, WiSARD e métodos de regressão alcança desempenho competitivo com ganhos expressivos em eficiência energética. Já [4] mostra que o desempenho varia significativamente com as condições do cenário, sendo afetado por oclusões, movimento irregular da câmera e variações de escala do alvo. A instabilidade do referencial da câmera, decorrente do balanço da embarcação, é um desafio recorrente nessas plataformas: em [11], um algoritmo de rastreamento para VSNT incorpora velocidade, arfagem e rumo, obtidos por GPS e unidade inercial, à atualização adaptativa dos filtros de correlação. Esse fenômeno é intrínseco ao domínio e reforça a importância de avaliar detectores sobre imagens adquiridas por uma plataforma real em movimento, e não sobre registros obtidos em condições estáticas.

No campo dos detectores de estágio único, [12] propõe a arquitetura YOLO original, que realiza detecção em tempo real a cerca de 45 quadros por segundo e serve de base conceitual para as versões avaliadas aqui. Comparações recentes entre versões da família concentram-se, contudo, em conjuntos de propósito geral: [13] avalia o YOLO26 frente ao YOLOv8 em Pascal VOC e VisDrone, sem cobrir o domínio marítimo nem quantificar o efeito da adaptação de domínio.

Quanto a conjuntos de dados, [14] evidencia a carência de bases anotadas voltadas à detecção de obstáculos por VSNT e contribui com o MaSTr1325, com 1.325 imagens anotadas em nível de pixel. Para detecção por caixas delimitadoras, o MODD2 [10] oferece 11.675 quadros estéreo a 1278×958 pixels, adquiridos por um VSNT real, cujos obstáculos foram

anotados manualmente pelos autores do conjunto e verificados por especialista. Nenhum desses conjuntos, porém, foi empregado para comparar detectores recentes sob protocolo único incluindo custo de inferência, lacuna que este trabalho endereça.

III. Metodologia

Esta seção descreve o procedimento adotado para comparar quatro detectores de estágio único na tarefa de detectar e localizar embarcações a partir de imagens embarcadas em um VSNT. A comparação é conduzida sob protocolo único, com os mesmos dados, a mesma partição, as mesmas anotações e as mesmas métricas, e considera dois eixos: o desempenho de detecção e o custo de inferência, que juntos determinam a escolha do modelo a ser embarcado. O código utilizado está disponível publicamente[1].

A. Conjunto de dados

Os experimentos utilizam o MODD2 [10], adquirido por um VSNT real em ambiente costeiro, com resolução de 1278×958 pixels. A Figura 1 ilustra uma cena típica de aquisição. O conjunto reúne 28 sequências de vídeo, totalizando 11.675 quadros na câmera esquerda, e acompanha anotações de referência produzidas pelos autores do conjunto: cada quadro foi rotulado manualmente e submetido a verificação por especialista, com cada obstáculo delimitado por uma caixa e a linha d'água é descrita por um polígono. Ao todo são 23.439 obstáculos anotados, e 22% dos quadros não contêm obstáculo algum, funcionando como exemplos negativos. Utiliza-se a versão retificada das imagens, com as anotações correspondentes. As anotações estão disponíveis apenas para a câmera esquerda, empregada neste trabalho; a câmera direita não é utilizada, de modo que todo o *ground truth* empregado provém da anotação original do MODD2, sem qualquer rótulo gerado automaticamente neste trabalho.

A tarefa é formulada como detecção de classe única: para cada imagem, o modelo deve indicar se há embarcação ou alvo flutuante e devolver a caixa delimitadora correspondente, em coordenadas de pixel. As anotações do MODD2 distinguem alvos *grandes*, cuja caixa cruza a linha d'água, de alvos *pequenos*, inteiramente contidos abaixo dela. Essa distinção funciona como indicador indireto de distância, e o recorte é central neste conjunto: a mediana da área das caixas corresponde a apenas 0,02% da imagem e 92% dos alvos ocupam menos de 1% dela, o que caracteriza o problema como predominantemente de detecção de alvos pequenos. Essa é a condição operacionalmente mais exigente, e aquela em que as arquiteturas mais se diferenciam.

Como quadros consecutivos de uma mesma sequência são quase idênticos entre si, utiliza-se um quadro a cada cinco, reduzindo o custo de treinamento sem perda relevante de informação e resultando em 2.335 imagens efetivamente empregadas. A partição entre treino, validação e teste é feita por sequência, e não por quadro: 20 sequências para treino, 4 para validação e 4 para teste. Assim, o conjunto de teste é composto por cenas que o modelo nunca viu, e as métricas medem generalização para novos cenários, e não apenas para novos quadros de uma cena já conhecida. Uma divisão por quadro produziria vazamento entre as partições e inflaria artificialmente os resultados.

[1]Repositório dos códigos utilizados no experimento, com endereço a ser incluído na versão final, após a aprovação da submissão.



Figura 1. Ambiente de aquisição: quadro da câmera esquerda registrado pelo VSNT em operação costeira. A embarcação aparece junto à linha do horizonte, condição em que ocupa poucas dezenas de pixels.

B. YOLOv8 e YOLO11

O YOLOv8 adota detecção *anchor-free* com *head* desacoplada, separando as ramificações de classificação e de regressão de caixas, e mantém a supressão de não-máximos (NMS) na saída [5]. O YOLO11 refina essa concepção com blocos de extração de características mais eficientes e mecanismos de atenção espacial, obtendo maior mAP com menor número de parâmetros [15]. Ambos são disponibilizados em variantes de tamanho crescente; neste trabalho emprega-se a variante *nano*, a de menor custo, por ser a compatível com as restrições de plataformas embarcadas.

C. YOLO26

O YOLO26 é a versão mais recente da família e introduz duas mudanças relevantes para o presente contexto: a operação *end-to-end* nativa, que dispensa a etapa de NMS, e a remoção da *Distribution Focal Loss* (DFL), simplificando a *head* de regressão [6]. A eliminação do NMS remove um pós-processamento de custo variável, o que é particularmente desejável em inferência embarcada, onde a previsibilidade da latência importa tanto quanto seu valor médio. O ganho de até 43% na velocidade de inferência atribuído a essas mudanças provém da documentação do próprio desenvolvedor [6], refere-se a execução em CPU e não decorre de avaliação independente. Medições de terceiros em conjuntos de propósito geral não confirmam vantagem consistente de latência sobre versões anteriores da família [13]. Este trabalho mede a latência em GPU, regime no qual o custo do NMS é diluído pelo

paralelismo e a alegação original, portanto, não é diretamente verificável.

D. SSD300

O SSD (*Single Shot MultiBox Detector*) prediz caixas delimitadoras e classes diretamente a partir de mapas de características em múltiplas escalas, empregando um conjunto fixo de âncoras por posição [7]. A variante utilizada, SSD300 com *backbone* VGG16, opera com entrada fixa de 300×300 pixels. Essa característica é determinante na comparação: enquanto os modelos YOLO processam a imagem em resolução bem superior, o SSD300 reduz drasticamente a área ocupada pelo alvo distante, o que tende a penalizá-lo justamente na condição mais difícil do conjunto. O SSD300 é incluído, portanto, como linha de base histórica de estágio único, e não como candidato equiparável às arquiteturas recentes.

E. Adaptação ao domínio por transfer learning

Os quatro modelos partem de pesos pré-treinados no COCO [8], que inclui a classe genérica *boat*. Avaliá-los diretamente nessa configuração fornece a linha de base e quantifica a diferença de domínio discutida na Seção III: o modelo conhece o conceito de embarcação, mas foi exposto a ângulos, escalas e fundos distintos dos encontrados na vista de um VSNT. A adaptação é feita por *transfer learning* [9], reaproveitando as representações visuais de baixo e médio nível aprendidas no COCO e reajustando as camadas de decisão para a classe única do problema. Nos modelos YOLO, o ajuste é aplicado à rede completa; no SSD300, a *head* de classificação é substituída por uma nova, com duas classes (fundo e alvo), antes do ajuste.

Do ponto de vista operacional, esse procedimento produz um modelo especializado no domínio de emprego da plataforma: iluminação, estado do mar, geometria de câmera e tipologia de embarcações da área de operação. O modelo resultante é o artefato efetivamente embarcado no VSNT, e o custo de produzi-lo, medido em tempo de treinamento, é reportado ao lado do custo de inferência.

F. Protocolo experimental

Todos os modelos são treinados e avaliados sobre a mesma partição, com a mesma semente e o mesmo conjunto de rótulos. Os modelos YOLO compartilham entre si número de épocas, resolução de entrada e tamanho de lote. O SSD300 segue o procedimento de treino de sua implementação de referência; por essa razão, seus resultados são interpretados como referência histórica, e não como medida isolada do mérito da arquitetura, ressalva retomada na Seção IV. A Tabela I reúne os parâmetros empregados; todos são comuns aos quatro modelos, com exceção da resolução de entrada, fixa em 300 px no SSD300. O custo computacional de cada arquitetura, incluindo o número de parâmetros, é retomado e comparado ao tempo de inferência na Seção IV (Tabela III).

O desempenho de detecção é reportado por precisão (P), revocação (R) e F1, calculados a partir das contagens de verdadeiros positivos (TP), falsos positivos (FP) e falsos negativos (FN):

Tabela I. Parâmetros experimentais utilizados. A partição é feita por sequência.

Parâmetro	Valor	Função
Épocas	20	Ciclos completos de treino
Resolução	768 px	Entrada YOLO (SSD300: 300 px)
Lote	8	Imagens processadas por iteração
Semente	42	Reprodutibilidade do experimento
Passo de quadros	1/5	Subamostragem da sequência
Confiança mínima	0,25	Limiar para aceitar detecção
IoU (NMS)	0,45	Supressão de redundâncias
IoU (avaliação)	0,50	Sobreposição mínima do acerto
Partição	70/15/15	Treino / validação / teste

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2\,P\,R}{P + R} \tag{1}$$

com limiar de interseção sobre união (IoU) de 0,5, complementados por mAP@50 e mAP@50-95, que não dependem da escolha de um limiar de confiança específico. As métricas finais são apuradas sobre a partição de teste, disjunta da partição usada para seleção de modelo, de modo a evitar viés otimista. O custo de inferência é medido na mesma plataforma para todos os modelos, descartando-se as dez primeiras execuções e registrando-se a média e o desvio-padrão das cinquenta seguintes.

IV. Resultados e Discussão

Esta seção apresenta: as condições experimentais, um exemplo qualitativo do comportamento dos detectores, o impacto quantitativo da adaptação ao domínio, a comparação entre arquiteturas, a eficiência de dados e as limitações do estudo.

A. Condições experimentais

Os experimentos foram executados no Google Colab, em GPU NVIDIA Tesla T4 de 16 GB, utilizando Ultralytics 8.4.126, PyTorch 2.11.0 e torchvision 0.26.0 (compilação com suporte a CUDA) e OpenCV 5.0.0. O conjunto de teste contém 285 imagens provenientes de 4 sequências não utilizadas em treino ou validação, totalizando 502 alvos anotados. Todos os modelos foram avaliados exatamente sobre esse conjunto, antes e depois do ajuste, com limiar de confiança de 0,25 e IoU de 0,5.

B. Análise qualitativa

A Figura 3 ilustra o comportamento dos detectores em um quadro com alvo distante, antes e depois da adaptação. Na condição pré-treinada, *nenhum* dos quatro modelos produz qualquer detecção sobre a embarcação: o alvo ocupa poucas dezenas de pixels e permanece abaixo do limiar de confiança, ainda que a classe *boat* exista no vocabulário do modelo. Após o ajuste, os quatro modelos passam a produzir detecções na mesma cena, cada um com duas caixas e confianças entre 59% e 94%, diferentemente do observado em execuções anteriores deste protocolo, nas quais um dos quatro modelos não detectava nenhum objeto nessa cena, reforçando a variabilidade entre sessões de treino discutida na Seção IV-F.

Observa-se ainda que os falsos positivos remanescentes concentram-se em duas situações recorrentes: reflexos especulares sobre a superfície da água, que produzem regiões claras

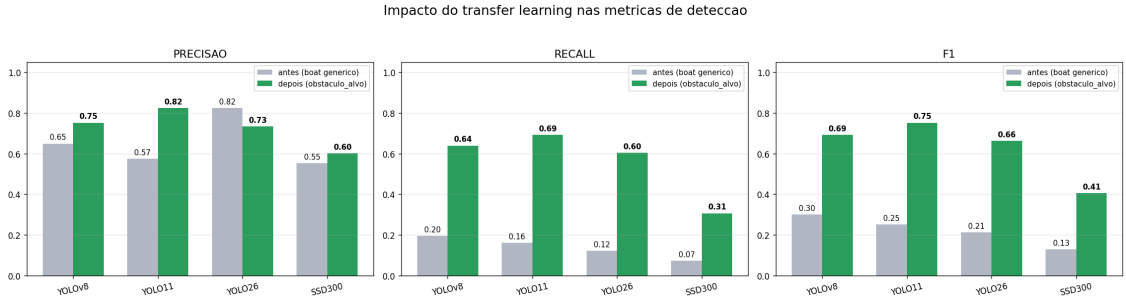


Figura 2. Precisão, revocação e F1 antes e depois da adaptação ao domínio.

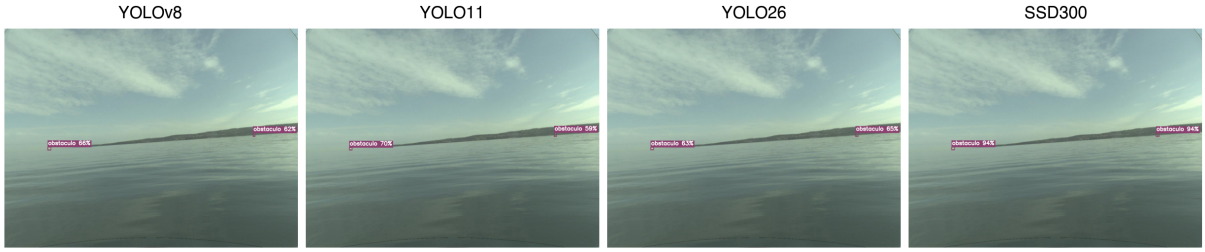


Figura 3. Detecções dos quatro modelos, após a adaptação ao domínio, em um quadro com alvo distante; antes do ajuste, nenhum dos quatro produzia qualquer detecção nessa cena. Confianças entre 59% e 94%.

de contorno definido, e estruturas costeiras próximas à linha do horizonte, cuja assinatura visual em baixa resolução aproxima-se da de uma embarcação distante. Ambas são características intrínsecas do domínio marítimo e não decorrem de deficiência de anotação.

C. Análise quantitativa: impacto da adaptação ao domínio

A Tabela II apresenta o desempenho antes e depois do ajuste. Os modelos pré-treinados no COCO, restritos à classe *boat*, alcançam F1 entre 0,130 e 0,300: a precisão é razoável, de 0,552 a 0,824, mas a revocação fica entre 0,074 e 0,195. Ou seja, quando esses modelos detectam algo, com frequência acertam; o problema é que deixam de detectar entre 80% e 93% dos alvos presentes. Esse padrão é a assinatura da diferença de domínio discutida na Seção III: o conceito de embarcação existe no vocabulário do modelo, mas a escala e o ponto de vista do domínio marítimo estão fora da distribuição de treinamento.

Após a adaptação, todos os modelos melhoram substancialmente, com ganhos de F1 entre $+0{,}265$ e $+0{,}493$. A Figura 2 evidencia onde essa melhora ocorre: o ganho concentra-se quase inteiramente na revocação, que cresce de três a cinco vezes, enquanto a precisão se mantém ou melhora. O ajuste não tornou os modelos permissivos. Tornou-os capazes de enxergar alvos que antes passavam despercebidos. A exceção é o YOLO26, único modelo cuja precisão regride (de 0,824 para 0,733). O comportamento é consistente com seu perfil conservador na condição pré-treinada, quando registrou apenas 13 falsos positivos: ao ganhar revocação, esse modelo troca parte da seletividade que o distinguia.

Tabela II. Desempenho antes e depois da adaptação ao domínio, no conjunto de teste (285 imagens, 502 alvos anotados).

Modelo	Estado	TP	FP	FN	Prec.	Rev.	F1
YOLOv8	antes	98	53	404	0,649	0,195	0,300
	depois	295	68	207	0,813	0,588	0,682
YOLO11	antes	81	60	421	0,574	0,161	0,252
	depois	**360**	**104**	**142**	**0,776**	**0,717**	**0,745**
YOLO26	antes	61	13	441	0,824	0,122	0,212
	depois	286	104	216	0,733	0,570	0,641
SSD300	antes	37	30	465	0,552	0,074	0,130
	depois	146	92	356	0,613	0,291	0,395

D. Análise comparativa entre arquiteturas

A Tabela III consolida a comparação após o ajuste, e a Figura 4 a apresenta no plano desempenho–latência. A tabela também reporta o número de parâmetros de cada arquitetura, permitindo avaliar se um modelo maior se traduz em melhor detecção. O YOLO11 obtém o maior mAP@50 (0,698) e o maior F1 (0,745) da comparação; o YOLOv8, por sua vez, obtém um mAP@50-95 marginalmente superior (0,352 contra 0,343). Em latência, contudo, o YOLO11 foi o mais lento dos quatro nesta execução (35,9 ms), resultado que inverte o observado em execuções anteriores deste mesmo protocolo, e que atribuímos à GPU alocada pelo provedor de nuvem a cada sessão, não a uma mudança de arquitetura (Seção IV-F). Não é possível, portanto, resolver aqui o compromisso entre precisão e latência: a escolha do modelo a embarcar exige nova medição em hardware fixo, preferencialmente já o de destino, e fica como trabalho futuro.

O SSD300 apresenta o pior desempenho da comparação, com F1 1,89 vez menor que o do YOLO11 apesar de possuir 9,2 vezes mais parâmetros. Dois fatores explicam o resultado: a entrada fixa de 300×300 pixels reduz um alvo de 34 pixels a

Tabela III. Comparação entre as arquiteturas após a adaptação ao domínio.

Modelo	Par. (M)	mAP@50	mAP@50-95	F1	ms
YOLOv8	3,01	0,667	**0,352**	0,682	$28{,}6 \pm 6{,}7$
YOLO11	2,58	**0,698**	0,343	**0,745**	$35{,}9 \pm 3{,}2$
YOLO26	2,38	0,621	0,316	0,641	$\mathbf{24{,}8 \pm 2{,}6}$
SSD300	23,75	0,318	0,159	0,395	$26{,}1 \pm 1{,}8$

menos de 8, abaixo do que a arquitetura consegue resolver; e o procedimento de treino segue a implementação de referência, distinta da receita compartilhada pelos modelos YOLO.

Um resultado consistente entre execuções merece destaque: o YOLO26, versão mais recente da família, não supera o YOLO11 em mAP nem F1, ficando 11,0% abaixo em mAP@50; observação semelhante, em domínio não marítimo, é reportada em [13]. Já em latência, a ordenação entre o YOLO26 e os demais variou entre sessões, reforçando a ressalva do parágrafo anterior.

A Tabela IV decompõe a revocação por faixa de tamanho do alvo, agora com o SSD300 avaliado pelo mesmo procedimento aplicado aos YOLO. Em objetos maiores (acima de 0,5% da imagem), os três YOLO empatam (0,985) e o SSD300 fica pouco atrás (0,955); na faixa intermediária, YOLOv8 e YOLO11 seguem empatados (0,948), à frente do YOLO26 (0,909) e do SSD300 (0,623). Toda a vantagem do YOLO11 no resultado agregado provém do regime de alvos muito pequenos, abaixo de 0,05% da imagem, onde alcança 0,618, contra 0,437 do YOLOv8, 0,421 do YOLO26 e apenas 0,097 do SSD300. Como essa faixa concentra 72% dos alvos do conjunto de teste, ela domina as métricas globais.

Tabela IV. Revocação por faixa de tamanho do alvo (IoU $\geq$ 0,5), no conjunto de teste. Entre parênteses, a quantidade de objetos em cada faixa.

Modelo	$<$0,05% (359 obj.)	0,05–0,5% (77 obj.)	$>$0,5% (66 obj.)
YOLOv8	0,437	**0,948**	**0,985**
YOLO11	**0,618**	**0,948**	**0,985**
YOLO26	0,421	0,909	**0,985**
SSD300	0,097	0,623	0,955

Por fim, o mAP@50-95 fica entre 47% e 51% abaixo do mAP@50 para os três modelos YOLO. Isso indica que os alvos são localizados, mas com caixas pouco justas. É o comportamento esperado quando o objeto ocupa poucas dezenas de pixels, pois um erro de dois ou três pixels já desloca significativamente o IoU.

E. Eficiência de dados

Para estimar quantos dados de treino são necessários, o YOLO11, o modelo de maior mAP@50, foi retreinado com frações crescentes do conjunto de treino, mantendo fixos os demais hiperparâmetros (Tabela V). O mAP@50 cresce de forma consistente com o volume de dados, sem sinal de saturação até 100% das imagens disponíveis, o que sugere que mais dados de treino ainda melhorariam o modelo.

F. Limitações

Seis limitações devem ser consideradas. Primeiro, o SSD300 opera com entrada fixa e segue receita de treino distinta da

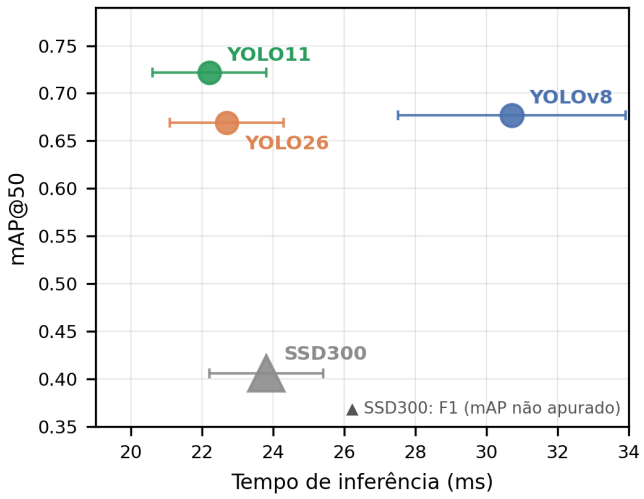


Figura 4. Compromisso entre desempenho de detecção e latência após a adaptação ao domínio. As barras horizontais indicam o desvio-padrão do tempo de inferência e o tamanho do marcador é proporcional ao número de parâmetros. Nesta medição, o YOLO11 é o mais preciso e também o mais lento dos quatro; a Seção IV-F discute a variabilidade da latência entre sessões.

Tabela V. Eficiência de dados: mAP@50 do YOLO11 por fração do conjunto de treino.

Fração	Imagens	mAP@50	mAP@50-95
25%	410	0,602	0,290
50%	821	0,633	0,317
75%	1232	0,668	0,349
100%	1642	0,698	0,343

dos modelos YOLO, de modo que sua desvantagem não é atribuível exclusivamente à arquitetura. Segundo, as medições de latência mostraram-se sensíveis à GPU alocada a cada sessão do Google Colab: execuções distintas deste mesmo protocolo produziram ordenações de velocidade diferentes entre os modelos, de modo que a comparação de latência aqui reportada deve ser confirmada em hardware fixo antes de orientar a escolha do modelo a embarcar. Terceiro, embora a partição por sequência garanta cenas de teste inéditas, as 28 sequências provêm de uma mesma região costeira e de uma mesma plataforma, o que limita a generalização para outras condições de mar e tipologia de embarcações. Quarto, as anotações do MODD2 cobrem obstáculos flutuantes, e não estruturas costeiras: embarcações atracadas em marinas, por exemplo, não são rotuladas. Detecções sobre esses objetos são contabilizadas como falsos positivos ainda que correspondam a embarcações reais, o que impõe um teto à precisão mensurável neste conjunto. Quinto, os modelos foram treinados sob teto fixo de 20 épocas e semente única, sem estimativa de variabilidade entre execuções de treino; diferenças inferiores a aproximadamente 0,02 nas métricas de detecção não devem ser interpretadas como vantagem. Sexto, os tempos de inferência foram medidos em GPU de classe datacenter, com envelope térmico e de potência muito superiores aos de uma plataforma embarcada; a verificação em hardware embarcado é trabalho futuro.

V. Conclusões

Este trabalho comparou quatro detectores de estágio único: YOLOv8, YOLO11, YOLO26 e SSD300, na detecção e localização de embarcações e alvos flutuantes em imagens adquiridas por um VSNT, sob protocolo experimental único e sobre anotações de referência do próprio conjunto, verificadas por especialista.

A adaptação ao domínio marítimo por *transfer learning* mostrou-se determinante: os ganhos de F1 variaram de $+0{,}265$ a $+0{,}493$, concentrados na revocação, que cresceu de três a cinco vezes. Modelos pré-treinados em conjuntos de propósito geral, ainda que dispondo da classe *boat*, deixam de detectar a maior parte dos alvos neste domínio.

Entre as arquiteturas avaliadas, o YOLO11 apresentou o maior mAP@50 (0,698) e o maior F1 (0,745) da comparação; o YOLO26, apesar de mais recente, não o superou em nenhuma dessas métricas. A decomposição por tamanho do alvo mostrou que a vantagem do YOLO11 provém especificamente do regime de alvos muito pequenos: nos objetos médios e grandes, os três modelos YOLO têm desempenho equivalente. Quanto à latência, a ordenação entre arquiteturas variou entre execuções deste mesmo protocolo, atribuído à GPU alocada a cada sessão de nuvem, não à arquitetura, de modo que o compromisso entre precisão e latência, decisivo para a escolha do modelo a embarcar, permanece em aberto e deve ser resolvido com medição em hardware fixo, preferencialmente já o de destino.

Como trabalhos futuros, pretende-se: avaliar as quatro arquiteturas diretamente sobre hardware embarcado, com quantização e otimização de grafo, resolvendo o compromisso precisão–latência apontado acima e orientando qual modelo de fato embarcar; estender a formulação para múltiplas classes, distinguindo embarcações de demais alvos flutuantes; testar estratégias de *data augmentation* mais agressivas, dado o volume limitado de dados; e explorar o par estéreo disponível no conjunto. A identificação de uma embarcação específica, tarefa complementar e de escopo distinto do aqui tratado, depende de um conjunto dedicado, com múltiplas imagens do mesmo alvo, hoje inexistente.

Referências

[1] S. C. C. da Silva, "O emprego de meios de superfície não tripulados," Passadiço, Rio de Janeiro, 2020.

[2] L. G. de Araújo, "Drones navais em ação: implicações nas operações militares contemporâneas," Monografia, Escola de Guerra Naval, Rio de Janeiro, 2024.

[3] A. G. Pereira, "Uma abordagem leve para detecção de embarcações utilizando redes neurais sem pesos e regressão," Dissertação (Mestrado em Engenharia de Sistemas e Computação), Universidade Federal do Rio de Janeiro, COPPE, Rio de Janeiro, 2025, acesso em: 23 abr. 2026. [Online]. Available: https://pesc.coppe.ufrj.br/uploadfile/publicacao/3252.pdf

[4] M. U. Din, A. B. Bakht, W. Akram, Y. Dong, L. Seneviratne, and I. Hussain, "Benchmarking vision-based object tracking for USVs in complex maritime environments," *IEEE Access*, vol. 13, pp. 15 014–15 025, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3532299

[5] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023. [Online]. Available: https://doi.org/10.3390/make5040083

[6] Ultralytics, "Ultralytics yolo26," Ultralytics Docs, 2026, acesso em: 7 ago. 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo26/

[7] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37. [Online]. Available: https://doi.org/10.1007/978-3-319-46448-0_2

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755. [Online]. Available: https://doi.org/10.1007/978-3-319-10602-1_48

[9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

[10] B. Bovcon, R. Mandeljc, J. Perš, and M. Kristan, "Stereo obstacle detection for unmanned surface vehicles by IMU-assisted semantic segmentation," *Robotics and Autonomous Systems*, vol. 104, pp. 1–13, 2018. [Online]. Available: https://doi.org/10.1016/j.robot.2018.02.017

[11] J. Jin, J. Zhang, D. Liu, J. Shi, D. Wang, and F. Li, "Vision-based target tracking for unmanned surface vehicle considering its motion features," *IEEE Access*, 2020. [Online]. Available: https://doi.org/10.1109/ACCESS.2020.3010327

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[13] C. G. Oguine, K. J. Oguine, O. M. Oguine, and O. C. Oguine, "YOLO26 vs. YOLOv8: A comprehensive architectural benchmark of next-generation real-time object detection models," *arXiv preprint arXiv:2605.24831*, 2026. [Online]. Available: https://arxiv.org/abs/2605.24831

[14] B. Bovcon, J. Muhovič, J. Perš, and M. Kristan, "The MaSTr1325 dataset for training deep USV obstacle detection models," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Macau, China, November 2019.

[15] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024. [Online]. Available: https://arxiv.org/abs/2410.17725